

A
PROJECT REPORT
on
FSM for Elevator Controller

Submitted in partial fulfilment of the requirements for the degree of
BACHELOR OF TECHNOLOGY



Under Guidance Of
Professor Pradeep C.
Head of Department
Electronics & Communication
Engineering

Submitted by
Harshit Mehta (15ETCEC008)
Praman Menaria (15ETCEC024)
Pawan Bagwan (15ETCEC022)
Prasun Gupta (15ETCEC300)

VIII-Sem Branch - ECE

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
TECHNO INDIA NJR INSTITUTE OF TECHNOLOGY, UDAIPUR-313001

2015-2019



Department of Electronics and Communication Engineering
Techno India NJR Institute of Technology, Udaipur-313001

Certificate

This is to certify that project work titled “FSM for Elevator Controller” by Harshit Mehta, Praman Menaria, Pawan Bagwan, Prasun Gupta was successfully carried out in the Department of Electronics and Communication Engineering, TINJRIT and the report is approved for submission in the partial fulfillment of the requirements for award of degree of Bachelor of Technology in Electronics and Communication. The work has been completed in all the respects during session 2015-2019.

Guided by

Dr. Vivek Jain

Assistant Professor

Electronics & Communication

Date.....

Head of the Department

Professor Pradeep Chhawchharia

Dept. of E.C.E TINJRIT, Udaipur

Date.....



Department of Electronics and Communication Engineering
Techno India NJR Institute of Technology, Udaipur-313001

Examiner Certificate

This is to certify that the following students Harshit Mehta, Praman Menaria, Prasun Gupta, Pawan Bagwan of final year B.Tech. (Electronics & Communication Engineering), was examined for the project work titled ***“FSM for Elevator Controller”*** during the academic year 2015 – 2019 at Techno India NJR Institute of Technology, Udaipur.

Remarks:

Date: _____

Signature

(Internal Examiner)

Name: -

Designation: -

Department: -

Organization: -

Signature

(External Examiner)

Name: -

Designation: -

Department: -

Organization: -

ACKNOWLEDGMENT

We take this opportunity to record our sincere thanks to all who helped us to successfully complete this work. Firstly, we are grateful to our **supervisor Dr. Vivek Jain** for his invaluable guidance and constant encouragement, support and most importantly for giving us the opportunity to carry out this work.

We would like to express our deepest sense of gratitude and humble regards to our **Head of Department Professor Pradeep Chhawchharia** for giving invariable encouragement in our endeavors and providing necessary facility for the same. We are very thankful to **Mr. Yogendra Singh Solanki, Assistant Professor** for encouraging us in this project work. Also, a sincere thanks to all faculty members of ECE, TINJRIT for their help in the project directly or indirectly.

Finally, we would like to thank my friends for their support and discussions that have proved very valuable for us. We are indebted to our parents for providing constant support, love and encouragement. We thank them for the sacrifices they made so that we could grow up in a learning environment. They have always stood by us in everything we have done, providing constant support, encouragement and love.

Harshit Mehta(15ETCEC008)

Praman Menaria(15ETCEC024)

Pawan Bagwan(15ETCEC022)

Prasun Gupta(15ETCEC300)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
TECHNO INDIA NJR INSTITUTE OF TECHNOLOGY, UDAIPUR-313001

Contents

- Acknowledgement
- Abstract
- Introduction
- Finite state Machine
- Design Strategy
- Verilog code
 - Input and Output pins
 - Register Parameters and Clock Generator
 - Initialization, Clock Generator Block & Request Floor Loop
 - Set three cases of elevator
 - Test Bench
- Simulation Results
 - Verilog Synthesis with RTL Compiler
 - Power Planning
 - Placement
 - Routing
 - Layout
 - DRC Result
 - Elevator Controller Design Symbol
- Conclusions

Abstract

With the advent of an era of staggering range of new technologies to provide ease of mobility and transportation elevators have become an essential component of all high rise buildings. An elevator is a type of vertical transportation that moves people between the floors of a high rise building. A four-Phase lift controller modeled on Verilog HDL code using Finite State Machine (FSM) has been presented in this paper. Verilog HDL helps in automated analysis and simulation of lift controller circuit. This design is based on synchronous input that operates on a fixed frequency. The Lift motion is controlled by means of accepting the destination floor level as input and generate control signal as output. In the proposed design a Verilog RTL code is developed and verified. Project Navigator of XILINX has been used as a code writing platform and results were simulated using Modelsim5.4a simulator. This project discusses the overall evolution of design and also discusses simulated results.

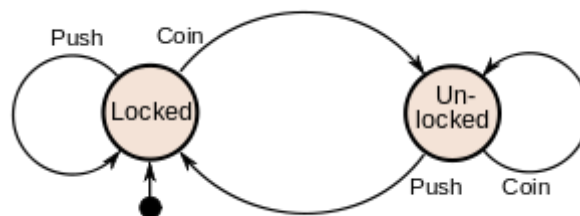
INTRODUCTION

This project is designed for an eight floor elevator controller of an integrated circuit that can be used as part of elevator controller. The elevator decides moving direction by comparing request floor with current floor. In a condition that the weight has to be less than 4500lb and door has to be closed in three minute. If the weight is larger than it, the elevator will alert automatically. The Door Alert signal is normally low but goes high whenever the door has been open for more than three minute. There is a sensor at each floor to sense whether the elevator has passed the current floor. This sensor provides the signal that encodes the floor that has been passed. The core parts of the design are shift register, three cases of elevator and the while loop when receive Request Floor.

Today's contemporary context calls for optimum intelligent usage of space. This requires the construction of high rise buildings in which the primary function of connectivity within the building and outside world is fulfilled by Elevators. They help in evacuating the buildings as soon as possible in case of any emergency. This project discusses the elevator controller design for vertical motion. Input buttons present on each floor outside the lift as well as inside the lift help the controller in servicing the floor calls. The lift controller using microprocessor based system is costlier. In this project it is proposed to design a low cost and compact controller modeled on Verilog HDL. Verilog is a Hardware Description Language used in designing of digital circuits by describing digital circuits in a textual manner. It has been used for the design and verification of the digital circuit at the Register Transfer Level of abstraction that allows the behavior of the required system to be described (modeled) and simulated. Project Navigator is the high-level manager for our design that modifies the source files in the workspace and runs process and the source file in the process window.

Finite State Machines

A **finite state machine** (sometimes called a finite state automaton) is a computation model that can be implemented with hardware or software and can be used to simulate [sequential logic](#) and some computer programs. Finite state automata generate [regular languages](#). Finite state machines can be used to model problems in many fields including mathematics, artificial intelligence, games, and linguistics.



A system where particular inputs cause particular changes in state can be represented using finite state machines. This example describes the various states of a turnstile. Inserting a coin into a turnstile will unlock it, and after the turnstile has been pushed, it locks again. Inserting a coin into an unlocked turnstile, or pushing against a locked turnstile will not change its state.

There are two types of finite state machines (FSMs): deterministic finite state machines, often called deterministic finite automata, and nondeterministic finite state machines, often called nondeterministic finite automata. There are slight variations in ways that state machines are represented visually, but the ideas behind them stem from the same computational ideas. By definition, deterministic finite automata recognize, or accept, [regular languages](#), and a language is regular if a deterministic finite automaton accepts it. FSMs are usually taught using languages made up of binary strings that follow a particular pattern. Both regular and non-regular languages can be made out of binary strings. An example of a binary string language is: the language of all strings that have a 0 as the first character. In this language, 001, 010, 0, and 01111 are valid strings (along with many others), but strings like 111, 10000, 1, and 11001100 (along with many others) are not in this language.

Design Strategy

In the coding part, we used several strategies to make the program works.

First, we defined the input and output current floor as `In_Current_Floor` and `Our_Current_Floor` to avoid same variable name as output and input.

Second, we add two more input pins - `Over_time` and `Over_Weight` in the code. These signals will be output from the mechanical machine to the controller. When the controller receives signal from weight alert or door alert, the complete will become one so that the elevator will stay unmoved at the `Out_Current_Floor`.

Third, define the `Out_Current_Floor`, `Direction`, `Complete`, `Door_Alert` and `Weight_Alert` as reg then assign them equal to the output. Therefore, those variables will run as a register and output.

Next, when the Reset is off the variable `Complete`, `Door Alert` and `Weight Alert` will be initialized to be zero. Similarly, when the `Request_Floor` is on, the variable `In_Current_Floor` is set to be equal to `Out_Current_Floor` only once.

Then, `In_Current_Floor` stay the same, `Out_Current_Floor` keep changing (updating) and compare with request floor, until `Out_Current_Floor` is at the same level as `Request_Floor`.

Lastly, define three cases of if statement for the elevator. There are cases for normal running cases – (comparing between `Request_Floor` and `Out_Current_Floor` to decide the moving direction), door open for more than three minutes - (turn on the `Door_Alert`) and overweight cases for elevator - (turn on the `Weight_Alert`).

Verilog Design Code

Verilog Code - Input and Output pins

```
module elevator_controller (request_floor,in_current_floor,clk,reset,complete,direction,
                           over_time,over_weight,weigh_alert,door_alert,out_current_floor);

//-----
//input pins
//-----
input [7:0]request_floor; // the 8 bit input request_floor;
input [7:0]in_current_floor;// the 8 bit input request_floor;
input clk; // we generate a low frequency clock
input reset; // the 1 bit input reset
input over_time; //the 1 bit input which indicates the door keep open for 3 minutes
input over_weight;// the 1 bit input which indicates the weight in the elevator is larger than 4500 lbs
//-----
//the output pins
//-----
output direction;// the 1 bit output which indicates the direction of the elevator
output complete;// the 1 bit output which indicates whether elevator is running or stopped
output door_alert;// the 1 bit output which indicates the door keep open for 3 minutes
output weigh_alert;// the 1 bit output which indicates the weight in the elevator is larger than 4500 lbs
output [7:0] out_current_floor;// the 8 bit ourput which shows the current floor
```

Input [7:0] Request_Floor

Define Request_Floor as 8-bit input variable. The 00000001 to 10000000 represent ground floor to eighth floor. (For example, second floor is 00000010, third floor is 00000100)

Input [7:0] In_Current_Floor

Define In_Current_Floor as 8-bit input variable. It represents the initial floor and the next complete floor (or the Out_Current_Floor that has reached the request floor).

Input clk

Define clk as input variable. To generate low frequency clock.

Input reset

Define reset as input variable. To generate reset.

Input Over_time

Define Over_time as input variable. In the Verilog code, "1" represents waiting time is longer than three minutes. Input Over_time will be the signal receives from timer (machinery part).

Input Over_Weight

Define Over_Weight as input variable. In the Verilog code, "1" represents elevator is overload. Input Over_Weight will be a signal receives from the weight detector (machinery part).

Output Direction

Define Direction as output variable. In the Verilog code, “1” represents moving up and “0” represents moving down.

Output Complete

Define Complete as output variable. In the Verilog code, “1” represents the elevator reaches the request floor. It can also represent pause (stay unmoved at the Out_Current_Floor), when the Over_Time or Weight_Alert is on.

Output Door_Alert

Define Door_Alert as output variable. In the Verilog code, “1” represents waiting time is longer than three minutes. Output Door_Alert will be on when Over_time is on.

Output Weight_Alert

Define Weight_Alert as output variable. In the Verilog code, “1” represents elevator is overload. Input Weight_Alert will on when Over_Weight is on.

Output [7:0] Out_Current_Floor

Define Out_Current_Floor as 8-bit output variable. It is a variable that will be used in the shift register.

Verilog Code – Register Parameters and Clock Generator

```
//-----  
//register parameters  
//-----  
reg r_direction;// 1 bit register connected to the output direction  
reg r_complete;// 1 bit register connected to the output complete  
reg r_door_alert;// 1 bit register connected to the output door_alert  
reg r_weigh_alert;// 1 bit register connected to the output weigh_alert  
reg [7:0] r_out_current_floor;// 8 bit register connected to the output out_current_floor;  
//-----  
//clock generator register  
//-----  
reg [12:0] clk_count;  
reg clk_200;  
reg clk_trigger;  
//-----  
//match pins and registers  
//-----  
assign direction=r_direction;  
assign complete=r_complete;  
assign door_alert=r_door_alert;  
assign weigh_alert=r_weigh_alert;  
assign out_current_floor= r_out_current_floor;  
//-----
```

Reg R_Direction

Define R_Direction as register parameter. In the Verilog code, “1” represents moving up and “0” represents moving down.

Reg R_Complete

Define R_Complete as register parameter. In the Verilog code, “1” represents the elevator reaches the request floor. It can also represent pause.

Reg R_Door_Alert

Define R_Door_Alert as register parameter. In the Verilog code, “1” represents moving up and “0” represents moving down.

Reg R_Weight_Alert

Define R_Door_Alert as register parameter. In the Verilog code, “1” represents elevator is overload.

Reg [7:0] R_Out_Current_Floor

Define R_Out_Current_Floor as register parameter. In the Verilog code, “1” represents moving up and “0” represents moving down.

Reg [12:0] CLK_Count

Define CLK_Count as 12-bit register parameter. It is a low frequency clock counter.

Reg CLK_200

Define CLK_200 as register parameter. It generates a 200 Hz clock.

Reg CLK_Trigger

Define CLK_Trigger as register parameter. It is the trigger of generator clock.

Assign Direction = R_Direction

To store the data of R_Direction into Direction

Assign Direction_Complete = R_Complete

To store the data of Direction_Complete into R_Direction

Assign Door_Alert = R_Door_Alert

To store the data of Door_Alert into R_Door_Alert

Assign Weight_Alert = R_Weight_Alert

To store the data of Weight_Alert into R_Weight_Alert

Verilog Code – Initialization, Clock Generator Block and Request Floor Loop

```
//initialization
//-----

always @ (negedge reset) begin

    clk_200=1'b0;
    clk_count=0;
    clk_trigger=1'b0;

//reset the clock registers
r_complete=1'b0; // set the default value to 0
r_door_alert=1'b0; //set the default value to 0
r_weigh_alert=1'b0; //set the default value to 0

end
//-----
```

This part will do the Initialization of the r_complete, r_door_alert and r_weight_alert.

Always @ (negedge reset)

It will run only when the reset is zero. This loop will reset *CLK_200*, *CLK_Count* and *CLK_Trigger* to be zero. Also, the *R_Complete*, *R_Door_Alert* and *R_Weeigh_Alert* will be set to be zero. If the reset is turn on to be one, the loop will not work.

```
// clock generator block
//-----

always @ (posedge clk) begin
    if(clk_trigger) begin
        clk_count=clk_count+1;
    end
    if(clk_count==5000)begin
        clk_200=~clk_200;
        clk_count=0;
    end
end
//-----
```

This is a clock generator that will run a loop when the *CLK* is on. The If Statement will determine whether the *CLK_Trigger* is on or the *CLK_Count* is equal to five thousand. If they are in the condition, the *CLK_Count* will increment every time the loop runs or the clock generator 200Hz will on. Therefore, the output control will run instantaneously.

```
//-----  
// in the case if there is a request floor  
//-----  
always @ (request_floor) begin  
    clk_trigger=1;  
    clk_200=~clk_200;  
  
    //trigger the clock generator  
    r_out_current_floor <= in_current_floor;  
  
    // put the value of the input in_current_floor to the register r_out_current_floor  
end  
//-----
```

This is the core part of the design.

It stores the data of *In_Current_Floor* to *R_Out_Current_Floor* only once.

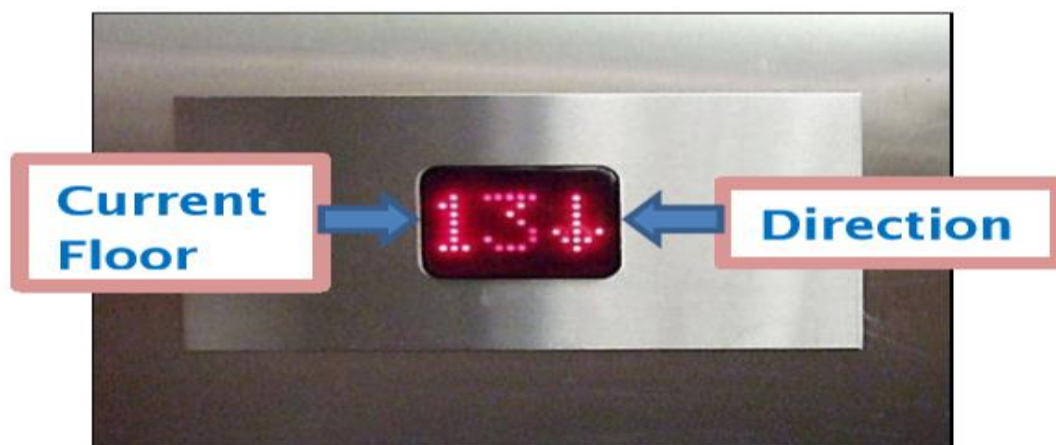
Always @ (request_floor)

The loop will run only when it receives request_floor.

It turn the *CLK_Trigger* on and trigger the 200Hz generator clock.

Also stores the data of *In_Current_Floor* to *R_Out_Current_Floor*.

Therefore, the *R_Out_Current_Floor* will be keep updating (or changing) whenever it reach the next floor, while the *In_Current_Floor* stay at the initial floor data.



Verilog Code – Set three cases of elevator

```
//-----  
//the normal running cases of the elevator  
//-----  
always @ (posedge clk) begin  
    if(!reset && !over_time && !over_weight)begin  
//-----  
// case 1: the normal running case of the elevator  
//-----  
        if (request_floor > r_out_current_floor) begin  
            r_direction=1'b1;  
            r_out_current_floor <= r_out_current_floor << 1;  
        end  
        else if (request_floor < r_out_current_floor) begin  
            r_direction= 1'b0;  
            r_out_current_floor = r_out_current_floor >> 1;  
        end  
        else if (request_floor == r_out_current_floor) begin  
            r_complete=1;  
            r_direction=0;  
        end  
    end  
end  
//-----
```

This part will do the three cases of elevator. There are cases for normal running, door overtime and over load.

Always @ (posedge clk)

This loop will run only when the *CLK* is on.

If (!reset && !over_time && !over_weight)

This loop will run only in the condition that *reset*, *over_time* and *over_weight* are off.

Case 1: the normal running case of elevator

If the *Request_Floor* is **Greater** than *R_Out_Current_Floor* the elevator will move up.

If the *Request_Floor* is **Smaller** than *R_Out_Current_Floor* the elevator will move down.

If the *Request_Floor* is **Equal** to *R_Out_Current_Floor* (it reach the request floor) the

R_Compelte is on and elevator stop moving

* The "<<1" and ">>1" are shift register. It shifts the data to the pointed direction by one.

For example, "<<1" will move "00000001" to "00000010"


```

//-----
//case 2: the door keep open for more than 3 minutes
//-----
    else if (!reset && over_time)begin
        r_door_alert=1;
        r_complete=1;
        r_weigh_alert=0;
        r_direction=0;
        r_out_current_floor <= r_out_current_floor;
    end
//-----
//case 3: the total weight in the elevator is more than 4500 lbs
//-----
    else if(!reset && over_weight)begin
        r_door_alert=0;
        r_weigh_alert=1;
        r_complete=1;
        r_direction=0;
        r_out_current_floor <= r_out_current_floor;
    end
end
endmodule

```

Case 2: the door has been open for more than three minute

If the *Reset* is off and *Over_Time* is on,

The *R_Door_Alert* and *R_Complete* are set to be ON

The *R_Weight_Alert* and *R_Direction* are set to be OFF

The *R_Out_Current_Floor* store its data to *R_Out_Current_Floor*

Therefore the door alert ring and the elevator will stop moving (or pause) when it is over time.

Case 3: the total weight in the elevator is more than 4500lb

If the *Reset* is off and *Over_Weight* is on,

The *R_Weight_Alert* and *R_Complete* are set to be ON

The *R_Door_Alert* and *R_Direction* are set to be OFF

The *R_Out_Current_Floor* store its data to *R_Out_Current_Floor*

Therefore the weight alert ring and the elevator will stop moving (or pause) when it is over load.

Verilog Test Bench Code

```
`timescale 1ns/1ns
module elevator_controller_tb;

    reg [7:0] request_floor;
    reg [7:0] in_current_floor;
    reg clk;
    reg reset;
    reg over_time;
    reg over_weight;

    wire direction;
    wire complete;
    wire door_alert;
    wire weigh_alert;
    wire [7:0] out_current_floor;

    elevator_controller elevator_controller_test(
        .request_floor(request_floor),
        .in_current_floor(in_current_floor),
        .clk(clk),
        .reset(reset),
        .direction(direction),
        .out_current_floor(out_current_floor),
        .complete(complete),
        .over_time(over_time),
        .over_weight(over_weight),
        .door_alert(door_alert),
        .weigh_alert(weigh_alert));
    //-----
    //generate the clock and test the circuit with different inputs
    //-----

    initial
        begin
            #0 clk = 1'b0; reset = 1'b1; over_time = 1'b0; over_weight = 1'b0;
            #50 reset = 1'b0;
            #50 reset = 1'b1;
            #50 request_floor = 8'b00000001; in_current_floor = 8'b10000000;
            #50 reset = 1;
            #50 reset = 0;
            #50 request_floor = 8'b00000001; in_current_floor = 8'b10000000;
            #50 reset = 1'b1;
            #50 reset = 1'b0;
            #50 over_time = 1;
            #50 reset = 1'b1;
            #50 reset = 1'b0;
            #50 over_weight = 1;
            #50 reset = 1'b1;
        end
    end
    always #50 clk = ~clk;
endmodule
```

In the Verilog Test Bench Code, after we defined reg and wire variables, we chose only two cases to test our code, because there are more than fifty possible cases.

The two cases we chose are:

Request floor = 00000001; In_Current_floor = 10000000

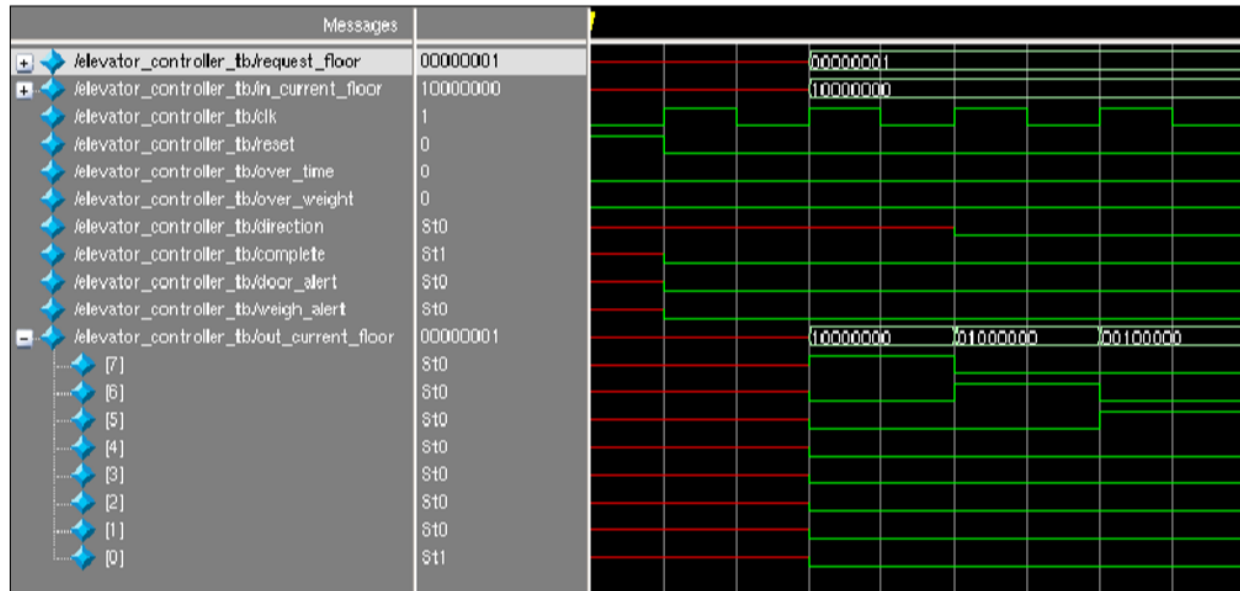
The elevator will move down from eighth floor to ground floor.

Request floor = 10000000; In_Current_floor = 00000001

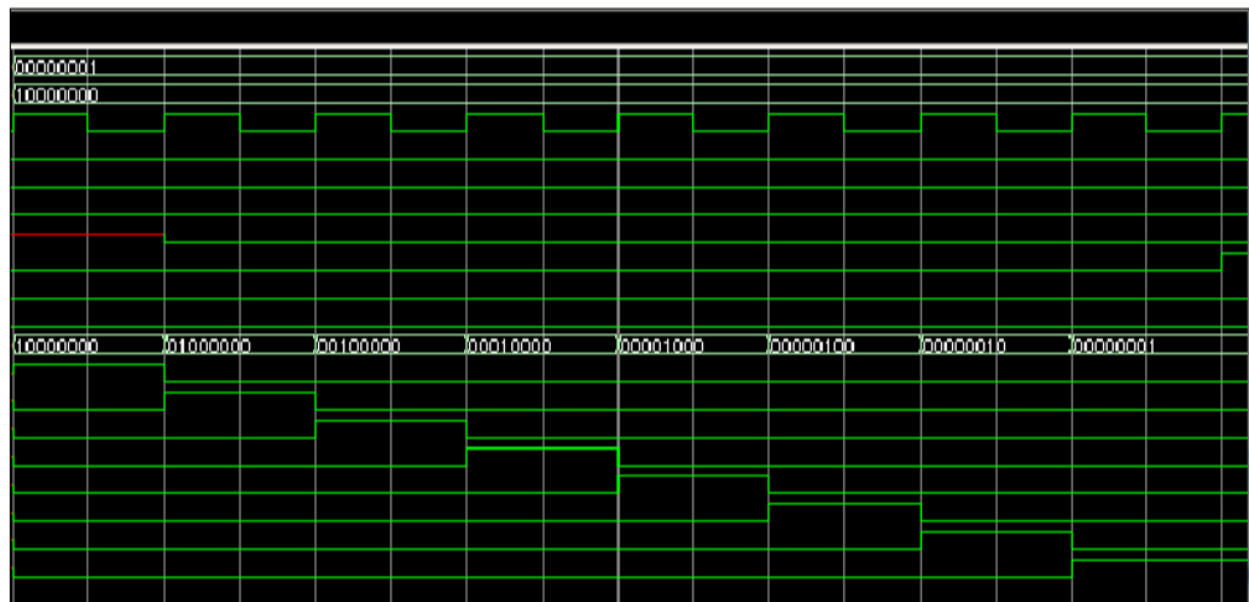
The elevator will move up from ground floor to eighth floor.

Simulation Results

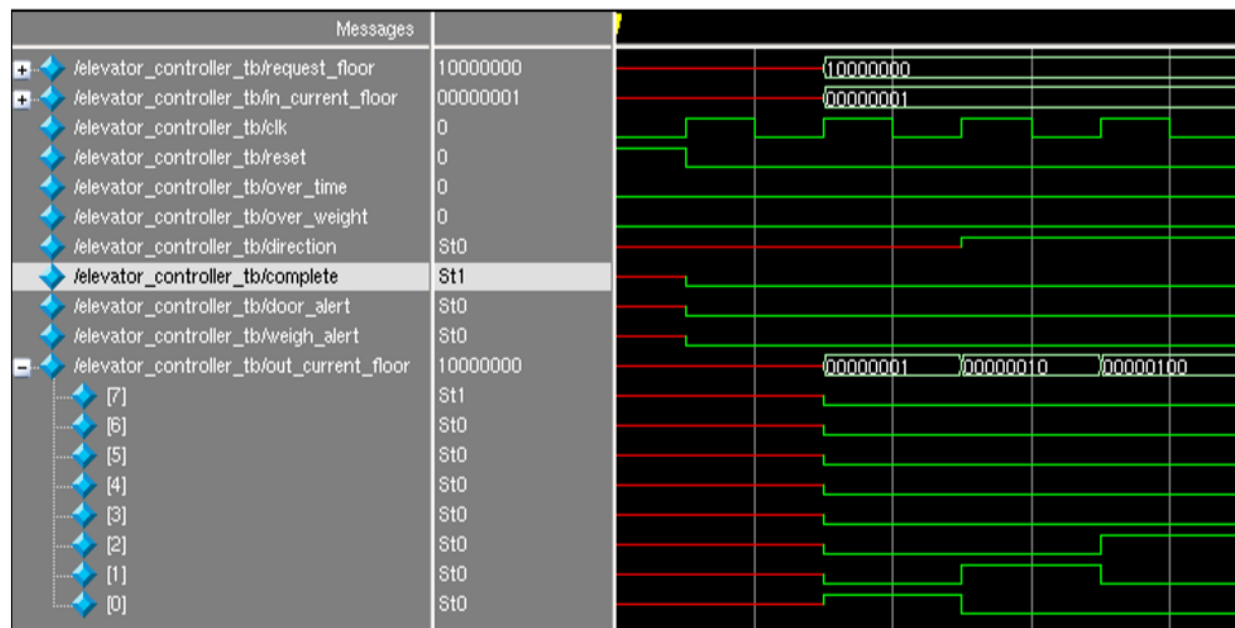
When Request floor = 00000001; In_Current_floor = 10000000



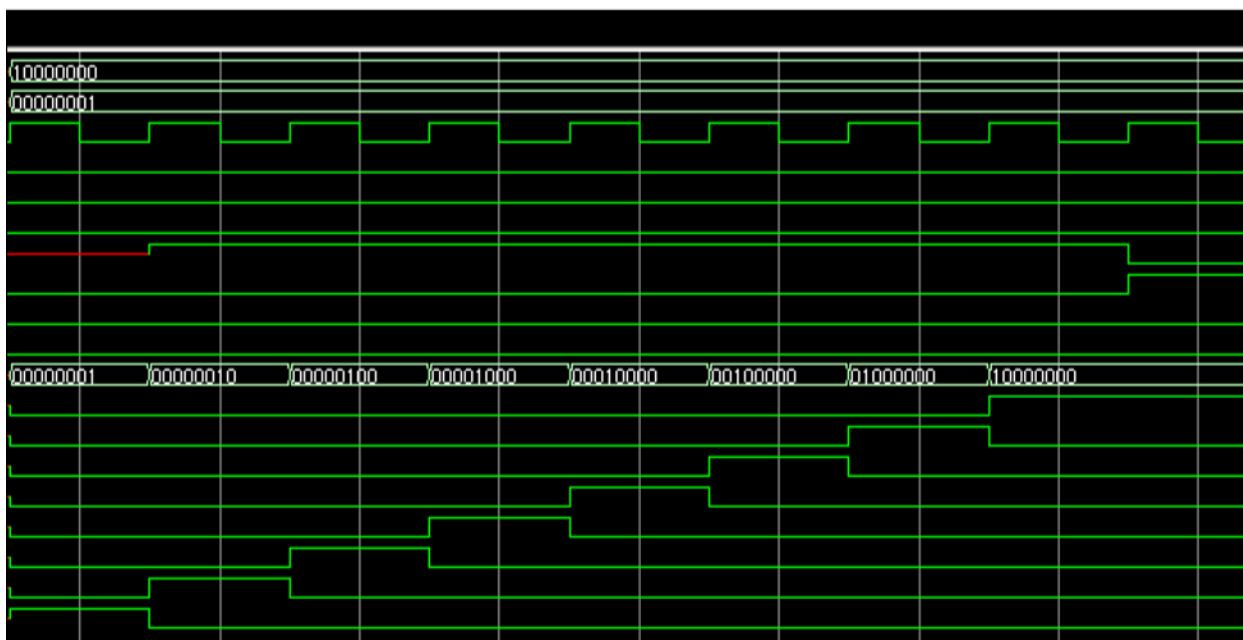
The elevator moves up from eighth floor to ground floor.



When Request floor = 10000001; In_Current_floor = 00000001

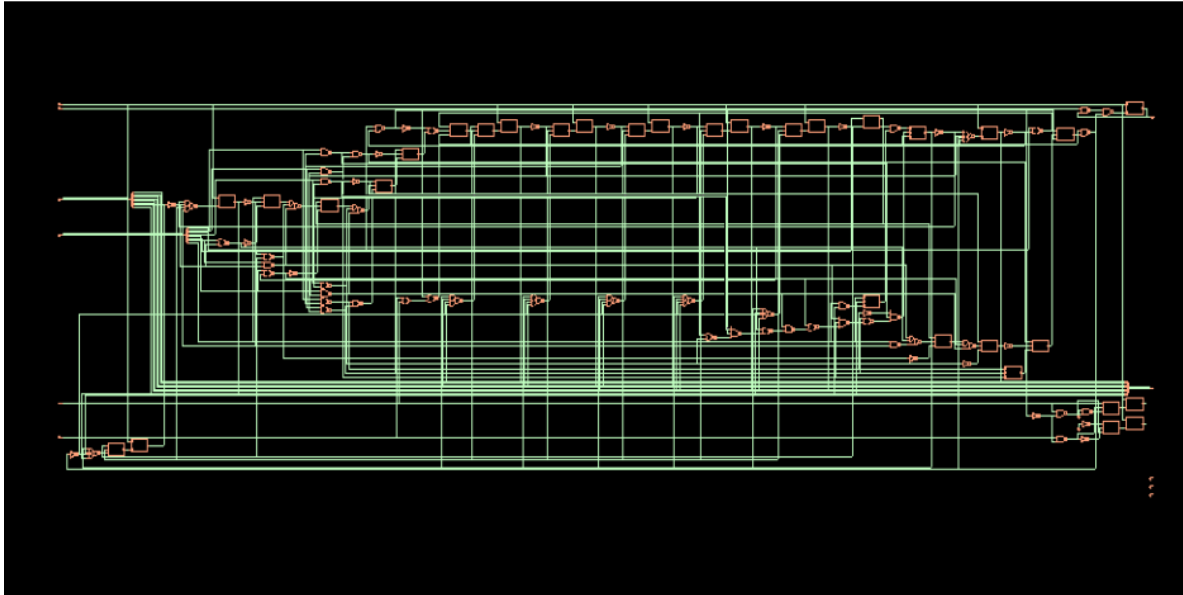


The elevator moves up from ground floor to eighth floor.

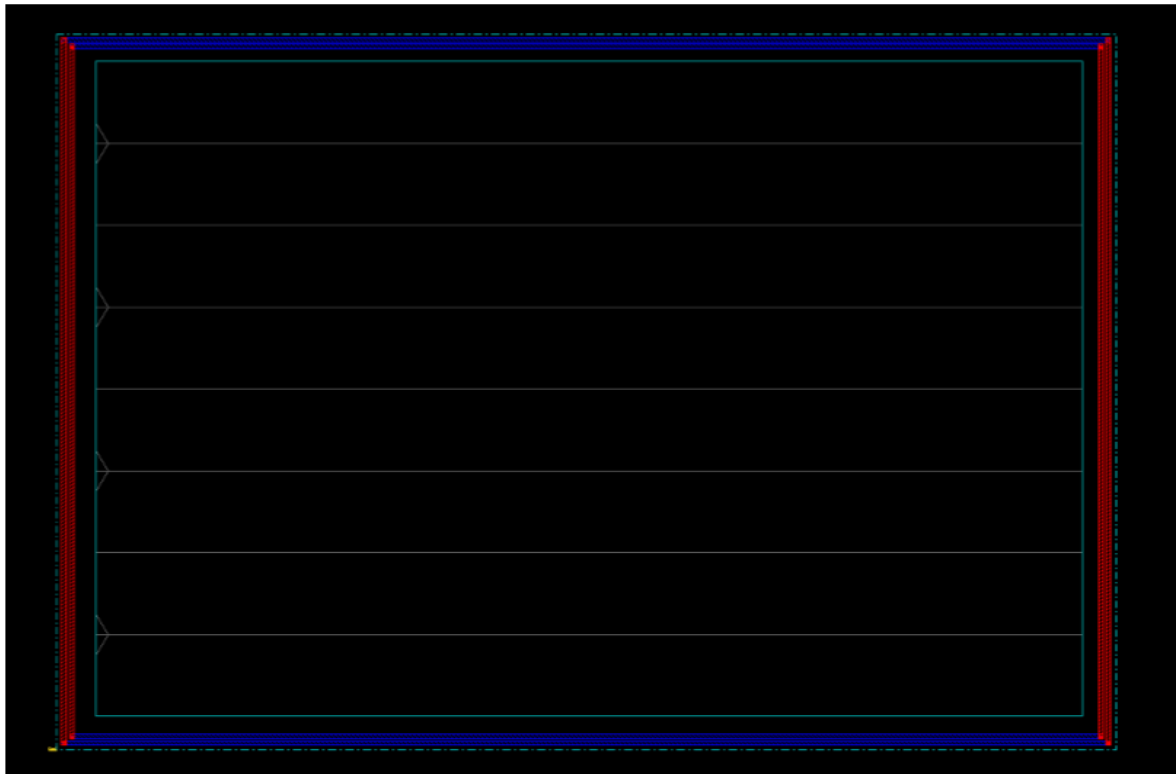


Schematic and Layout

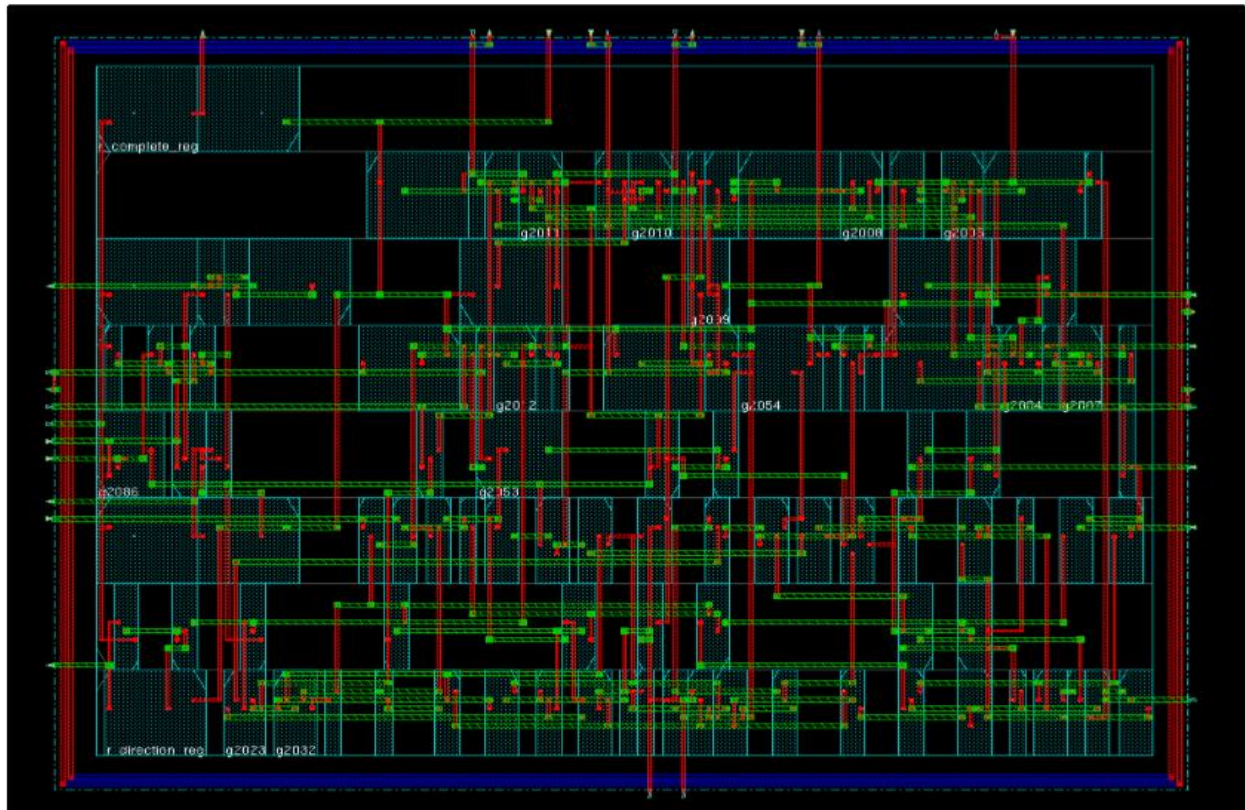
Verilog Synthesis with RTL Compiler



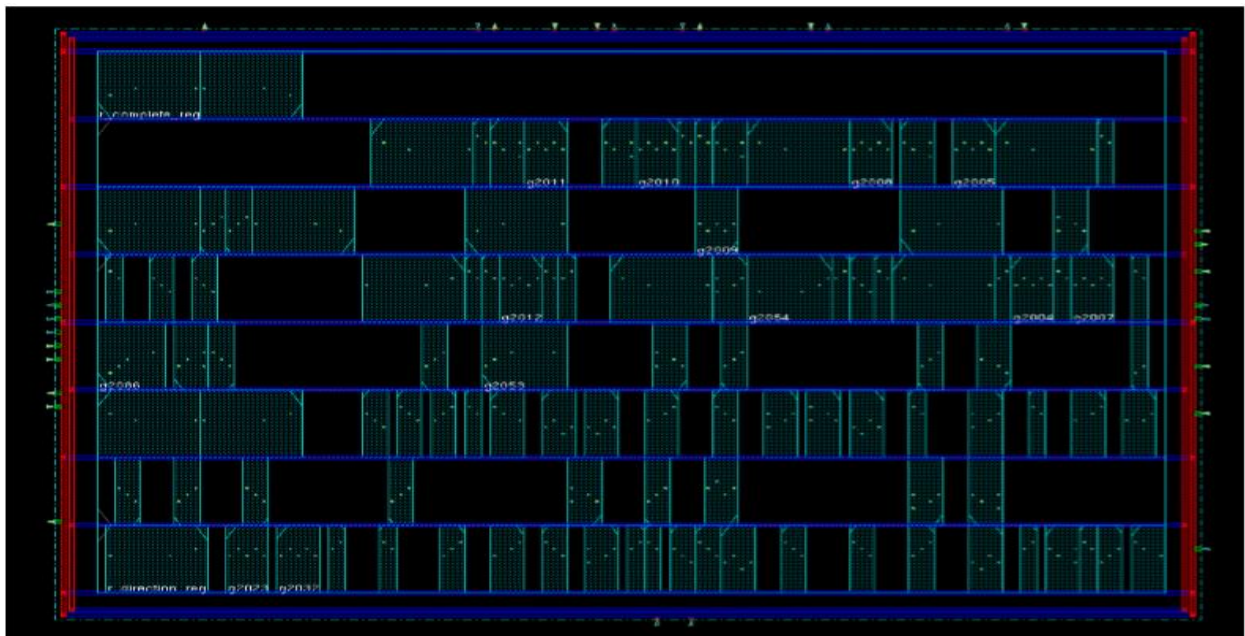
Power Planning



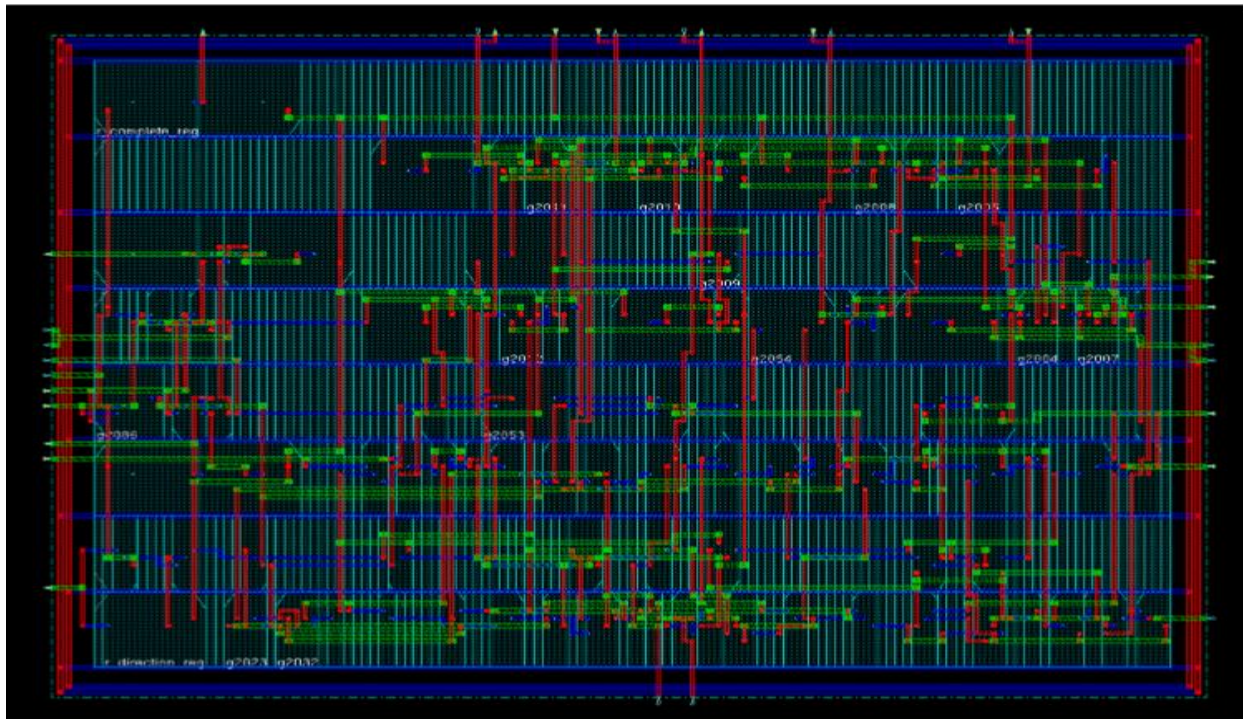
Placement



Routing



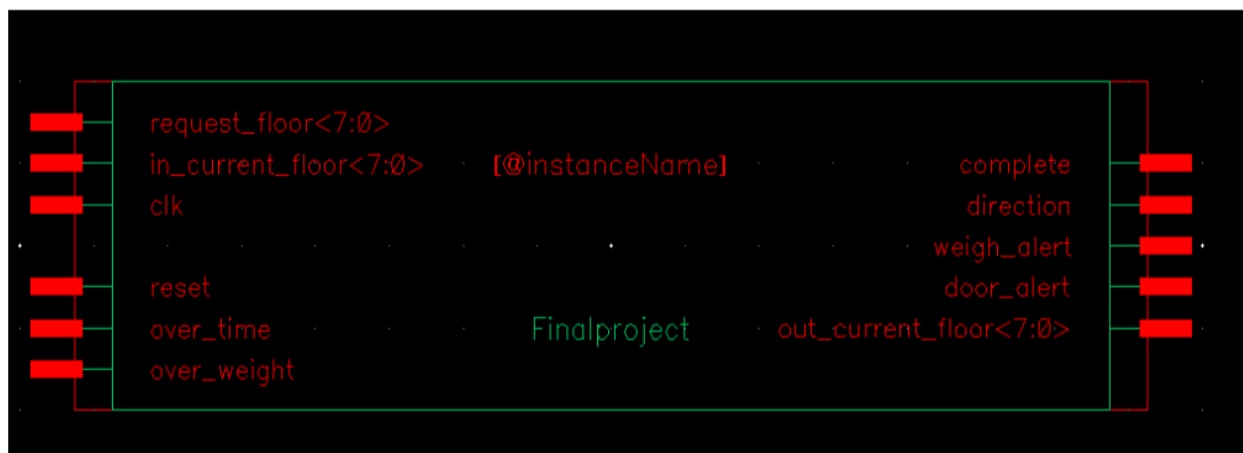
Filler Cells



DRC Result

```
File Tools Options Help cadence
executing: saveDerived(gemAnd(highres ca) errMesg)
executing: saveDerived(gemAnd(highres cp) errMesg)
executing: drc(highresEdge activeEdge (sep < (lambda * 2.0)) errMesg)
executing: drc(highresEdge gemGetEdge(gemAndNot(elec gemButting(elec elecHighres))) (sep < (...
executing: saveDerived(gemButting(elecHighres gemAndNot(elec elecHighres) (ignore == 2)) errM...
executing: saveDerived(gemAnd(elecHighres nwell) "(SCMOS Rule 27.6) resistor must be outside w...
executing: saveDerived(gemAnd(elecHighres active) "(SCMOS Rule 27.6) resistor must be outside ...
executing: drc(elecHighresEdge (width < (lambda * 5.0)) errMesg)
          drc(elecHighresEdge (sep < (lambda * 7.0)) errMesg)
          drc(elecHighresEdge (notch < (lambda * 7.0)) errMesg)
executing: drc(highresEdge elecHighresEdge (enc < (lambda * 2.0)) errMesg)
DRC started.....Sun Apr 28 12:10:23 2013
  completed ....Sun Apr 28 12:10:25 2013
  CPU TIME = 00:00:00  TOTAL TIME = 00:00:02
***** Summary of rule violations for cell "Finalproject layout" *****
  Total errors found: 0
```

Elevator Controller Design Symbol



Conclusion

For this project, we learned the basic idea of how does the normal elevators run in many cases, even though it is simplified, we still spent lots of time to design and to figure out many problems when combining all the cases, of course the most challenging and time consuming part is debugging. However, after accomplishing it, we learned many things beyond this project, so it is a very helpful assignment.